



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

GELU Module Design

Department of Electrical Engineering
IIT Hyderabad

Team members:

Moka Rochishnu	– EE25MTECH11009 –(FPGA)
Pippari Rohith	– EE25MTECH12013 –(Verification)
Manthri Arjun Kumar	– EE25MTECH14013 –(RTL)
Pillala Jagadish Sai Kumar	– EE25MTECH14012 –(RTL)

ABSTRACT

The Gaussian Error Linear Unit (GELU) activation function is widely used in modern deep learning architectures such as Transformers and BERT models due to its smooth nonlinear characteristics. However, direct hardware implementation of GELU is computationally expensive because of exponential and hyperbolic tangent operations. This project presents an FPGA-based hardware implementation of the GELU activation function using fixed-point arithmetic and lookup table approximation techniques. The design was implemented in Verilog HDL and synthesized on an FPGA platform. Simulation and synthesis results demonstrate reduced hardware complexity, lower latency, and efficient resource utilization while maintaining acceptable computational accuracy.

Keywords— GELU, FPGA, Fixed-point arithmetic, Q8.8 representation, Verilog HDL, LUT approximation

INTRODUCTION

Artificial Neural Networks (ANNs) are widely used in modern Artificial Intelligence (AI) applications such as image processing, speech recognition, and natural language processing. In neural networks, activation functions are important because they introduce nonlinearity and help the network learn complex patterns from data. Common activation functions like Sigmoid and Tanh suffer from problems such as vanishing gradients, while the Rectified Linear Unit (ReLU) is simple and fast but may cause dead neuron problems.

To overcome these limitations, the Gaussian Error Linear Unit (GELU) activation function was introduced. GELU is widely used in advanced deep learning models such as Transformer and BERT architectures developed by companies like OpenAI and Google. Compared to ReLU, GELU provides smoother output and improves the learning capability and accuracy of neural networks.

The mathematical expression of GELU is:

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right)$$

Field Programmable Gate Arrays (FPGAs) are suitable for implementing neural network hardware because they provide high speed, parallel processing, and reconfigurable architecture. In this project, the GELU activation function is implemented on FPGA using Verilog HDL with fixed-point arithmetic and approximation techniques such as Lookup Tables (LUTs). The proposed design reduces hardware complexity and improves computational speed while maintaining good accuracy. This implementation can be useful for AI accelerators and real-time embedded systems.

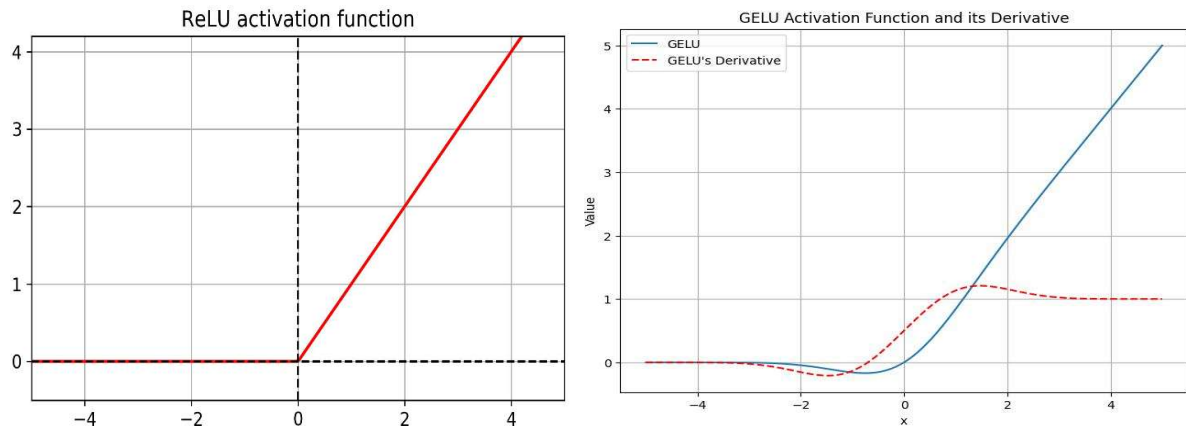
Comparison of Activation Functions:

Activation Function Advantages		Limitations
Sigmoid	Smooth output	Vanishing gradient
Tanh	Zero-centered output	Gradient saturation
ReLU	Simple and fast	Dead neuron problem
GELU	Smooth activation and better accuracy Higher hardware complexity	

Why GELU?

Unlike ReLU, GELU does not completely remove negative inputs. Instead, it smoothly scales the inputs based on their importance. This improves gradient flow, convergence speed, and model accuracy.

Modern AI models such as Transformer and BERT architectures use GELU activation because of its superior performance.



Unlike ReLU, which removes all negative values, GELU provides probabilistic activation by smoothly scaling the inputs. The mathematical equation of GELU is:

$$\text{GELU}(x) = 0.5x(1 + \tanh[\sqrt{2/\pi} (x + 0.044715x^3)])$$

The equation consists of:

- Cubic operation (x^3)
- Multiplication
- Hyperbolic tangent function
- Final scaling operation

Problem Statement:

The GELU activation function provides better accuracy and learning capability in deep learning models. However, direct hardware implementation is computationally expensive because it involves:

- Multiplication operations
- Cubic computations
- Hyperbolic tangent calculations

- Floating-point arithmetic

These operations increase hardware complexity, power consumption, and latency.

Therefore, an optimized FPGA implementation using fixed-point arithmetic and LUT approximation is required to achieve high speed and efficient resource utilization.

PROPOSED HARDWARE ARCHITECTURE

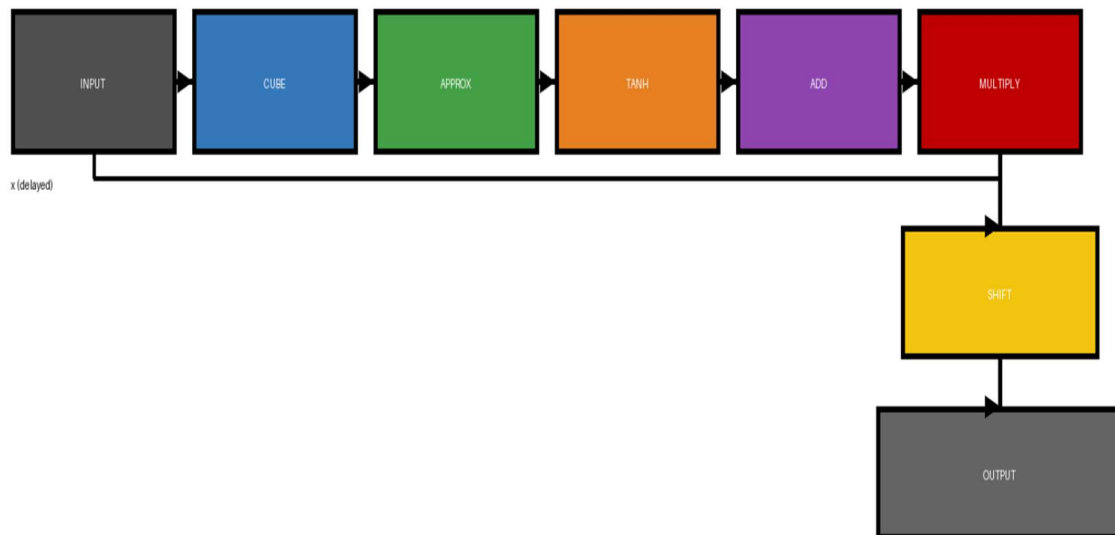
The proposed hardware architecture implements the Gaussian Error Linear Unit (GELU) activation function using a 16-bit Q8.8 fixed-point representation on FPGA. The architecture is designed using Verilog HDL and optimized for efficient hardware utilization by replacing complex mathematical operations with pipelined arithmetic and lookup table (LUT)-based approximation methods.

Overall Architecture & Work Flow

The complete GELU hardware architecture consists of four major modules:

1. Cubic Computation Module
2. GELU Approximation Module
3. tanh Lookup Table (LUT) Module
4. Final GELU Output Module

The design follows a pipelined architecture to improve throughput and reduce computation delay.



Xcube Module:

```
// xcube module
module signed_cube_q8_8 (
    input wire clk,
    input wire rst,
    input wire signed [15:0] x,    // Q8.8
    output reg signed [15:0] y    // Q8.8
);
    // Intermediate pipeline registers
    reg signed [15:0] x_d1,x_d2;
    reg signed [31:0] x_sq;    // Q16.16
    reg signed [47:0] x_cb;    // Q24.24

    always @(posedge clk) begin
        if (rst) begin
            x_d1 <= 0;
            x_d2 <= 0;
            x_sq <= 0;
            x_cb <= 0;
            y <= 0;
        end else begin
            x_d1 <= x;
            x_d2 <= x_d1;
            // Stage 1:  $x^2$ 
            x_sq <= x * x;
            // Stage 2:  $x^3$ 
            x_cb <= x_sq * x_d2;
            // Scale back to Q8.8 (truncate 16 LSBs)
            // y <= $signed(x_cb[31:16]);
            y <= x_cb >>> 16;
        end
        // $display("x=%f x_cube=%f", x/256.0, x_cb/16777216.0);
    end
endmodule
```

Tanh Module:

```
// tanh module
module tanh_lut_q8_8 (
    input wire      clk,
    input  signed [15:0] x_in,    // Q8.8
    output reg signed [15:0] y_out // Q8.8
);
    wire signed [15:0] x_clamped;

    // Clamp to [-4, +4] -> [-1024, +1024]
    assign x_clamped = (x_in > 16'sd1024) ? 16'sd1024 :
                      (x_in < -16'sd1024) ? -16'sd1024 :
                      x_in;

    // Scale to index
    wire [7:0] index;
    //assign index = (x_clamped + 16'sd1024) >>> 3;
    assign index = ((x_clamped + 16'sd1024) >>> 3) > 255 ?
                  8'd255 :
                  ((x_clamped + 16'sd1024) >>> 3);

    // LUT
    reg signed [15:0] lut [0:255];
    initial begin
        $readmemh("tanh_q8_8.mem", lut);
    end
    always @(posedge clk) begin
        y_out <= lut[index];
    end
endmodule
```

Gelu Approx module: Approx. value of $(\sqrt{2/\pi}) * (x + 0.044715 * x^3)$

```
module gelu_approx_q8_8 (
    input wire clk,
    input wire rst,
    input wire signed [15:0] x, // Q8.8
    output reg signed [15:0] y // Q8.8
);
    // Constants in Q8.8
    localparam signed [15:0] K1 = 16'sd204; // sqrt(2/pi) * 256
    localparam signed [15:0] K2 = 16'sd11; // 0.044715 * 256
    wire signed [15:0] x_cube;
    // Delayed versions of x for alignment
    reg signed [15:0] x_d1, x_d2, x_d3;
    // Intermediate registers
    reg signed [31:0] mul1; // K2 * x^3
    reg signed [15:0] sum; // x + K2*x^3
    reg signed [31:0] mul2; // final multiply
    // Instantiate cube module
    signed_cube_q8_8 cube_inst (
        .clk(clk),
        .rst(rst),
        .x(x),
        .y(x_cube)
    );
    always @ (posedge clk) begin
        if (rst) begin
            x_d1 <= 0;
            x_d2 <= 0;
            x_d3 <= 0;
            mul1 <= 0;
            sum <= 0;
            mul2 <= 0;
            y <= 0;
        end else begin
            // Delay x to match cube latency (3 cycles)
            x_d1 <= x;
            x_d2 <= x_d1;
            x_d3 <= x_d2;
            mul1 <= x_cube * K2; // Q8.8 * Q8.8 = Q16.16 // Stage 1: 0.044715 * x^3
            sum <= x_d3 + $signed(mul1[23:8]); // Scale back to Q8.8
            mul2 <= sum * K1; // Q8.8 * Q8.8 = Q16.16 // Stage 2: multiply with sqrt(2/pi)
            y <= $signed(mul2[23:8]); // Final scaling back to Q8.8
        end
    end
endmodule
```

Final GELU Module:

```
// final gelu module
module gelu_final_q8_8 (
    input wire clk,
    input wire rst,
    input wire signed [15:0] x,    // Q8.8
    output reg signed [15:0] out   // Q8.8
);

    // Intermediate signals
    wire signed [15:0] y;          // from gelu_approx
    wire signed [15:0] tanh_y;     // from LUT

    // Delayed x to align pipeline
    reg signed [15:0] x_d1, x_d2, x_d3, x_d4, x_d5, x_d6, x_d7;

    // Intermediate registers
    reg signed [15:0] one_plus_tanh;
    reg signed [31:0] mul1, mul2;

    // Constant 1 in Q8.8
    localparam signed [15:0] ONE = 16'sd256;    // 1.0

    // Instantiate GELU approx (gives y)
    gelu_approx_q8_8 gelu_inst (
        .clk(clk),
        .rst(rst),
        .x(x),
        .y(y)
    );

    // Instantiate tanh LUT
    tanh_lut_q8_8 tanh_inst (
        .clk(clk),
        .x_in(y),
        .y_out(tanh_y)
    );

    always @(posedge clk) begin
        if (rst) begin
            x_d1 <= 0; x_d2 <= 0; x_d3 <= 0; x_d4 <= 0; x_d5 <= 0; x_d6 <= 0; x_d7 <= 0;
            one_plus_tanh <= 0;
            mul1 <= 0;
            mul2 <= 0;
            out <= 0;
        end else begin
            // Delay x to match pipeline
            x_d1 <= x;
            x_d2 <= x_d1;
            x_d3 <= x_d2;
            x_d4 <= x_d3;
            x_d5 <= x_d4;
            x_d6 <= x_d5;
            x_d7 <= x_d6;
            // Stage 1: (1 + tanh(y))
            one_plus_tanh <= ONE + tanh_y;    // Q8.8
            // Stage 2: x * (1 + tanh(y))
            mul1 <= x_d7 * one_plus_tanh;    // Q8.8 * Q8.8 = Q16.16
            // Stage 3: multiply by 0.5 -> divide by 2
            mul2 <= mul1 >>> 1;

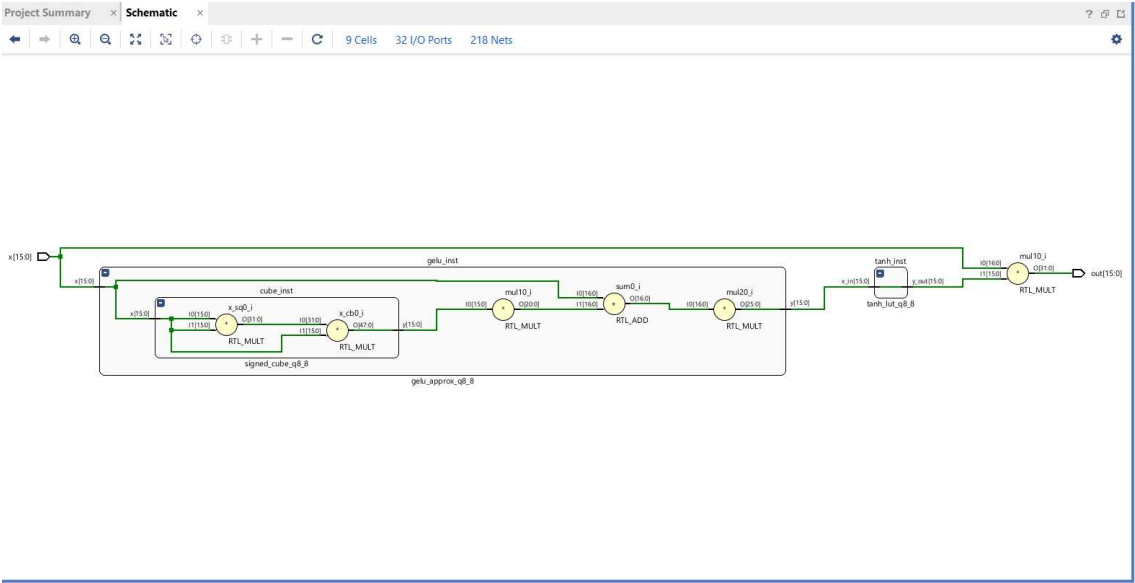
            // Scale back to Q8.8
            if (mul2[31:24] != 0 && mul2[31:24] != 8'hFF)
                out <= (mul2[31]) ? -16'sd256 : 16'sd255;    // for +ve overflow, clamping, out =255, for -ve OF out=-256
            else
                out <= $signed(mul2[23:8]);
            // out <= mul2[23:8]; this is truncation, overflow not considered
        end
    end
endmodule
```

Synthesis Report:

```
161 Detailed RTL Component Info :
162 +---Adders :
163     2 Input    32 Bit    Adders := 1
164     2 Input    16 Bit    Adders := 3
165 +---Registers :
166         48 Bit    Registers := 1
167         32 Bit    Registers := 1
168         16 Bit    Registers := 16
169 +---Multipliers :
170         16x32    Multipliers := 1
171 +---Muxes :
172     2 Input    16 Bit    Muxes := 3
173     2 Input    9 Bit    Muxes := 1
174     2 Input    8 Bit    Muxes := 1
---
```

```
378
379 Report Cell Usage:
380 +-----+
381 |      |Cell| |Count|
382 +-----+
383 |1| |BUFG| | 1|
384 |2| |DSP48E1| | 1|
385 |3| |LUT2| | 16|
386 |4| |LUT4| | 2|
387 |5| |LUT6| | 16|
388 |6| |SRL16E| | 16|
389 |7| |FDRE| | 52|
390 |8| |IBUF| | 18|
391 |9| |OBUF| | 16|
392 +-----+
393
394 Report Instance Areas:
395 +-----+
396 |      |Instance| |Module| |Cells|
397 +-----+
398 |1| |top| | | | 138|
399 |2| | gelu_inst | |gelu_approx_q8_8| | 19|
400 |3| | cube_inst | |signed_cube_q8_8| | 2|
401 +-----+
402
403 Finished Writing Synthesis Report : Time (s): cpu = 00:00:23 ; elapsed = 00:00:27 . Memory (MB): peak = 1107.012 ; gain = 692.750
404
```

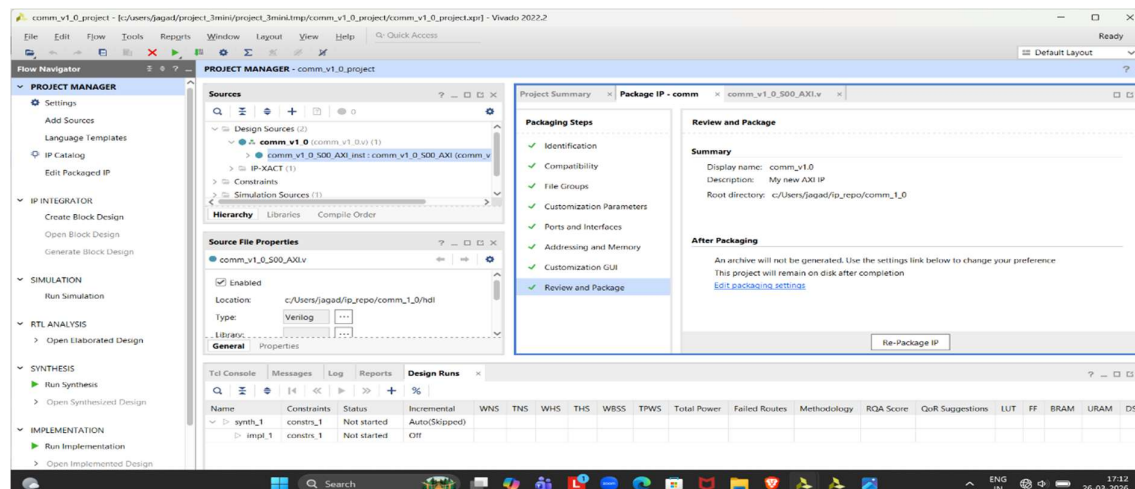
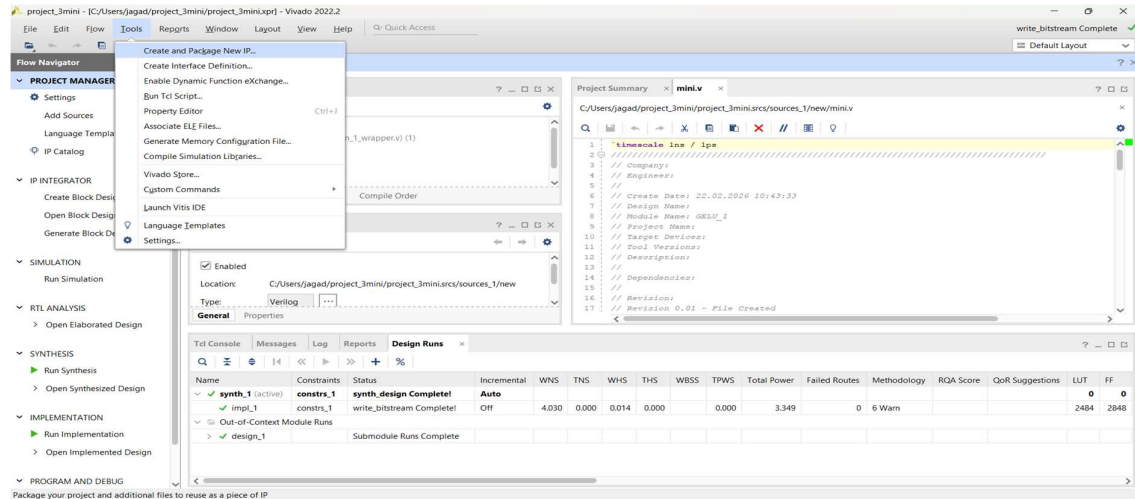
Schematic (Block design):



FPGA implementation

After the completion of RTL code and verifying testbench go to tools and click Create and Package New Ip

"Packaging" refers to the process of turning your HDL design (Verilog, VHDL, or SystemVerilog) into a reusable IP (Intellectual Property) core. Packaging allows you to map individual ports of your HDL code to Interface Standards (like AXI4, AXI-Lite, or AXI-Stream).



Block Design:

1. ZYNQ Ultra Scale+ MPSoC IP

- It represents the Processing System (PS). It handles the high-level software, memory controllers (DDR), and standard peripherals (USB, Ethernet, etc.).

2. comm_0 (Custom IP)

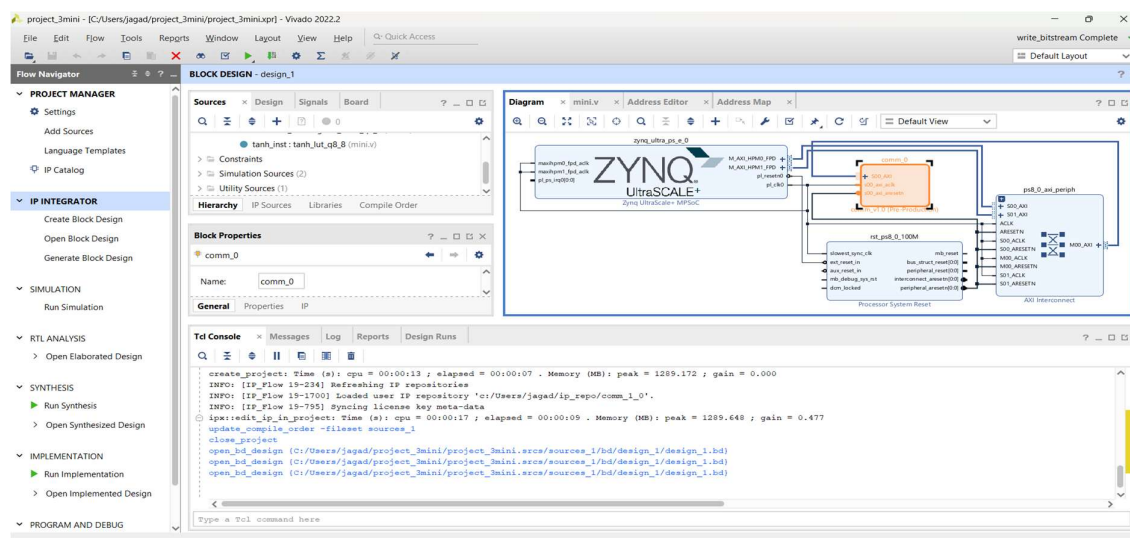
- It is a custom IP block (likely an RTL module you packaged).
- It is connected to the AXI Interconnect, meaning the processor views this block as a memory-mapped peripheral.

3. AXI Interconnect (ps8_0_axi_periph)

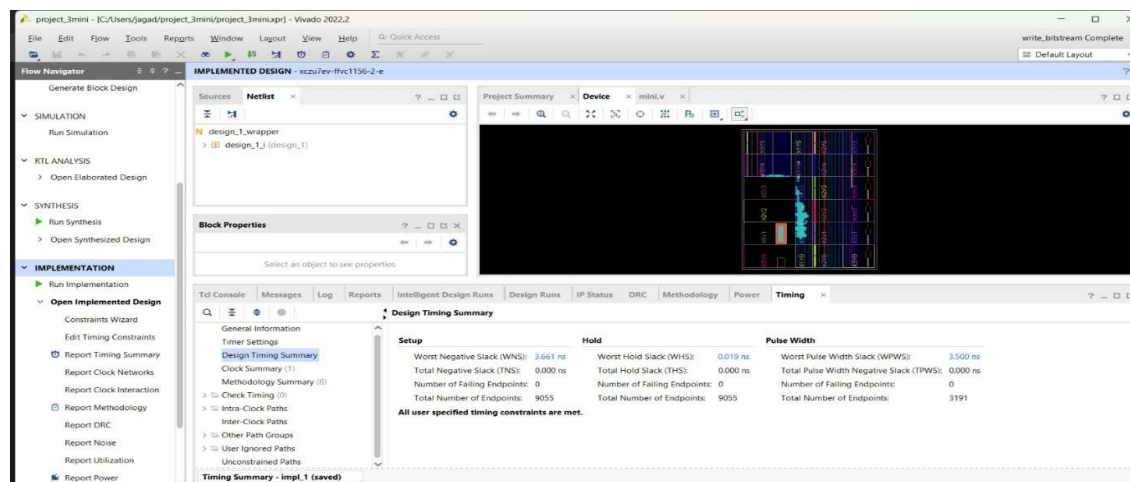
- It takes the commands from the Zynq PS and routes them to the correct peripheral (like your comm_0 block).

4. Processor System Reset

- It synchronizes the reset signals for the entire system.
- It ensures that when the chip powers up or resets, the Zynq PS, the Interconnect, and comm_0 block all start at the exact same time and don't get stuck in an undefined state.
- comm block is likely very fast at processing math (like the tanh function), but it doesn't have a built-in way to store gigabytes of data or talk to the outside world (like an Ethernet port or an SD card).
- The PS writes a value to a Register inside comm block via the AXI bus.
- Hardware logic sees that register change and updates its behaviour instantly.



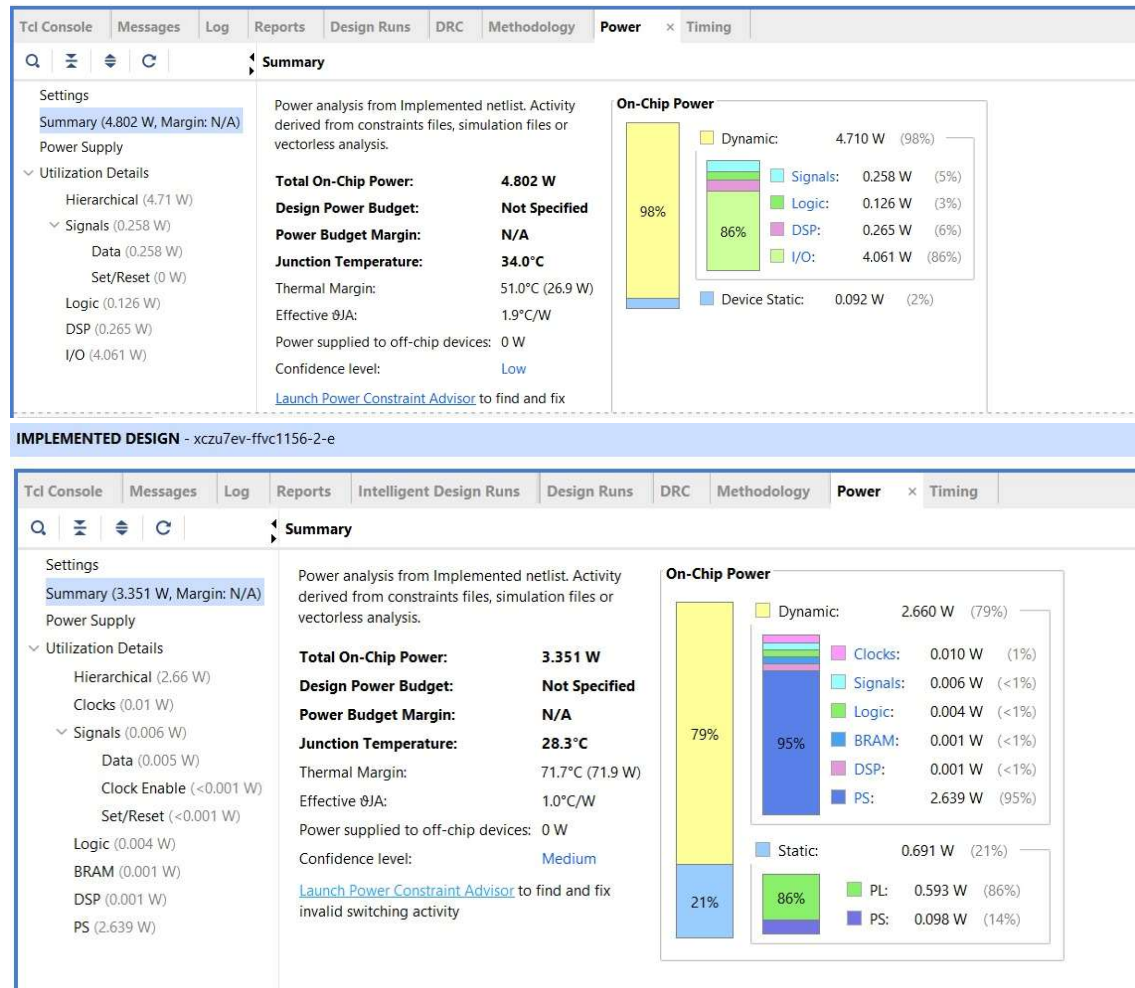
Reports after generation of bitstream



Worst negative slacks should be positive.

Negative Setup: Data is too slow. Slow down the clock or optimize logic.

Negative Hold: Data is too fast. Add delay to the data path (handled by Vivado).



Vitis Process flow

After the bitstream generation check of timing slacks and power reports export the hardware including bitstream and launch Vitis IDE.

Create platform project and application project and build them respectively.

1. Platform Project

The Platform Project is the foundation. It tells the software exactly what the physical hardware looks like.

- **Source:** It is built from the .xsa file, exported from Vivado.
- **What it contains:** Board Support Package (BSP): The drivers that allow the code to talk to your comm_0 block, the UART (serial terminal), and the clocks.

- Role: It tells the compiler how to communicate with specific FPGA design.

2. Application Project

The Application Project is the brain.

- Source: Written in C or C++.
- What it contains:

The Xil_In32 and Xil_Out32 commands (seen in your code) used to send data to your hardware block.

Even though "Hello World" doesn't use comm_0 block, it validates the entire communication path from your computer to the silicon. Hence this program was run as standalone initially before Gelu code in vitis.

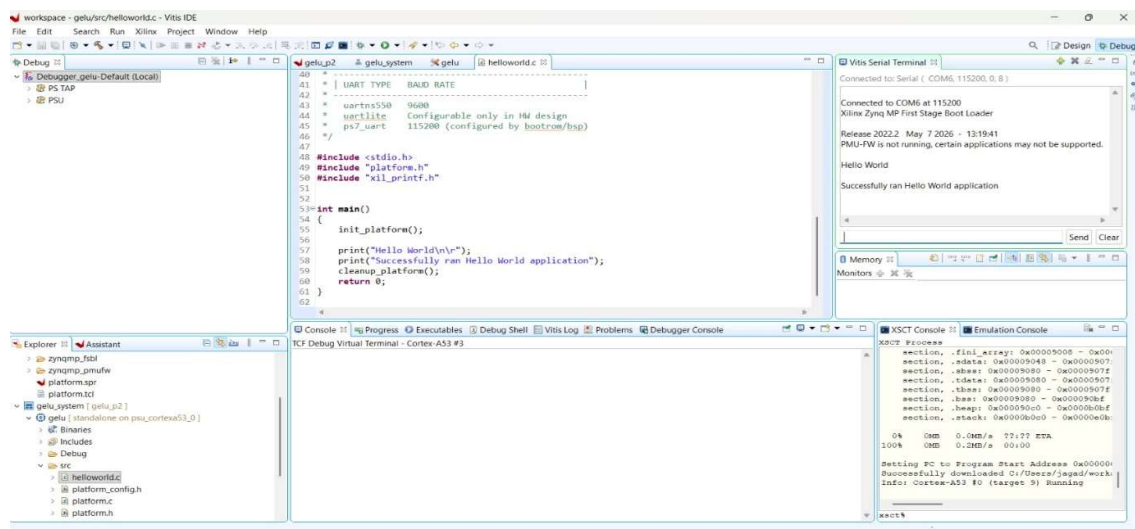
1. The "UART" Path (The Voice)

- The Zynq PS is alive and executing code.
- The physical USB-to-UART cable between the FPGA board and your PC is working.
- The Baud Rate (115200) is set correctly. If this didn't work, GELU results cannot be seen.

2. The Boot & Programming Sequence

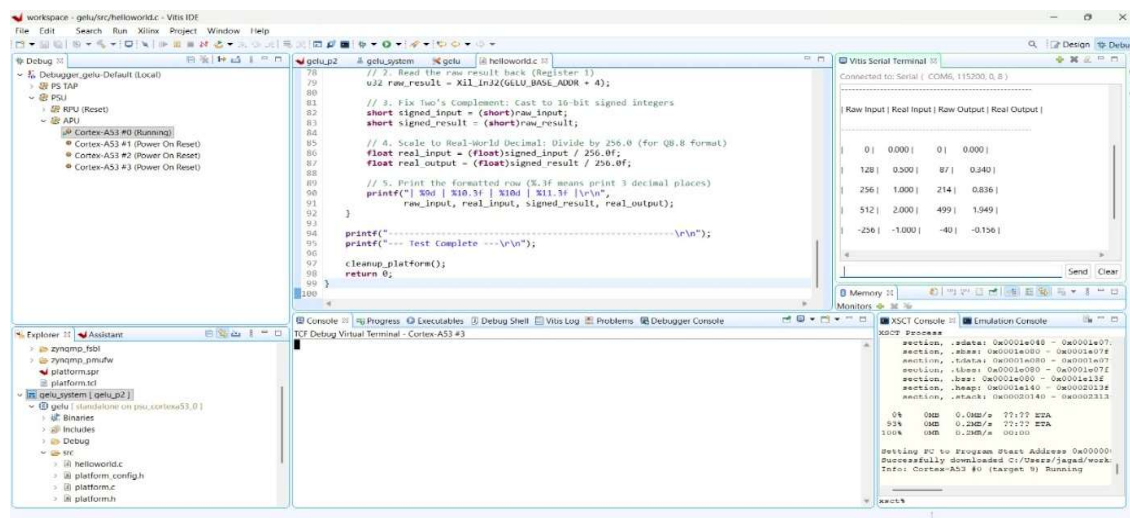
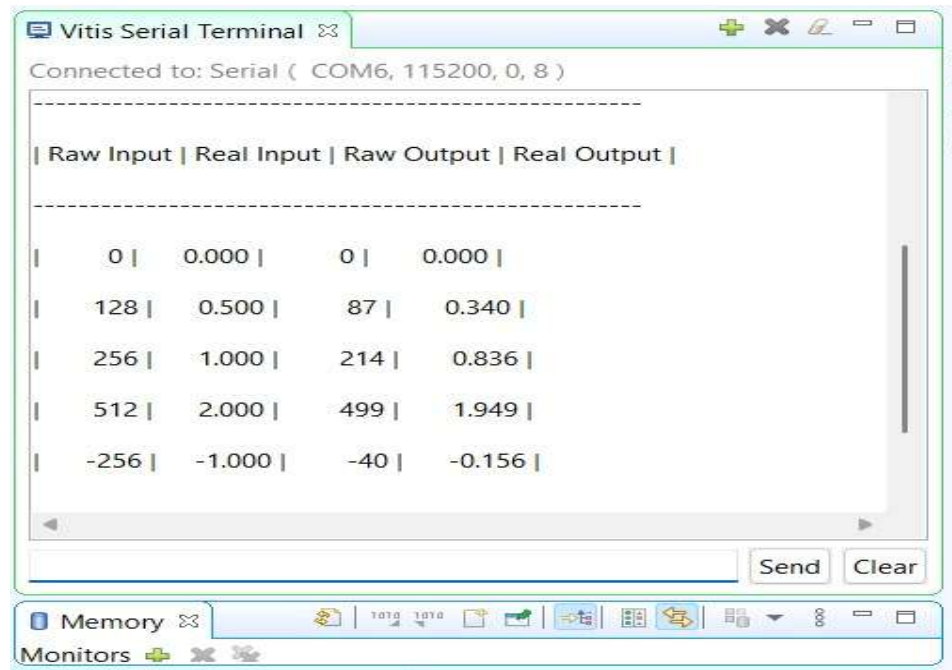
- To see "Hello World," Vitis had to:
- Initialize the JTAG connection.
- Download the Bitstream (hardware) to the FPGA.
- Download the Executable (software) to the ARM Processor.

If "Hello World" runs, it means Hardware Wrapper is valid and the chip is accepting the code. If this fails, GELU code has zero chance of running.



Once hello world program is programmed then modify hello world code and write the inputs for GELU code and right click on .c file and click on run configuration->> launch hardware and check the results in Vitis serial terminal.

Output Results



Verification of Fixed-Point GELU Using Python

This report explains the verification of a fixed-point GELU RTL design using Python. The RTL design was implemented using Verilog HDL and verified using a Python golden model. The verification compares the RTL-equivalent output with the mathematical GELU function.

Python verification method

Python is used as a golden reference model. The Python code computes:

- Mathematical GELU output
- RTL-equivalent fixed-point GELU output
- Error between both outputs
- Graphical comparison plots

Why did I print Only Certain Values?

The Python code prints values only near $x = 0$ and $x = 1$ using:
if $\text{abs}(x) < 0.1$ or $\text{abs}(x-1) < 0.05$

Reason:

- Around $x = 0$, GELU changes from negative to positive values.
- Small errors become visible near transition region.
- Around $x = 1$, GELU output becomes strongly positive.
- These points help verify whether the RTL approximation behaves correctly

Command Prompt Output Analysis

The command prompt output shows comparison between RTL output and true GELU output.

Example:

$x = -0.040$ | RTL = -0.01953 | TRUE = -0.01955 | ERROR = 0.00002

Observation:

- RTL output is almost equal to true GELU output.
- Error is extremely small.
- This confirms correct behavior near transition region.

Another Example:

$x = 1.010$ | RTL = 0.85547 | TRUE = 0.85214 | ERROR = 0.00333

Observation:

- Positive input region is also accurately approximated.
- Error is still very small.
- This confirms good accuracy of fixed-point RTL implementation.

Figure 1: Command Prompt Verification Results

```
x = -0.040 | RTL = -0.01953 | TRUE = -0.01955 | ERROR = 0.00002
x = 0.040 | RTL = 0.01953 | TRUE = 0.02085 | ERROR = -0.00132
x = 1.010 | RTL = 0.85547 | TRUE = 0.85214 | ERROR = 0.00333

Verification Results
-----
Maximum Error = 0.0033254987568986927
Mean Square Error = 1.7817915943165902e-06
|
```

Important points:

1. RTL output and true GELU output are very close.
2. Errors are extremely small.
3. Maximum error is approximately 0.0033.
4. Mean Square Error (MSE) is very low.
5. These results confirm that the RTL approximation behaves correctly.

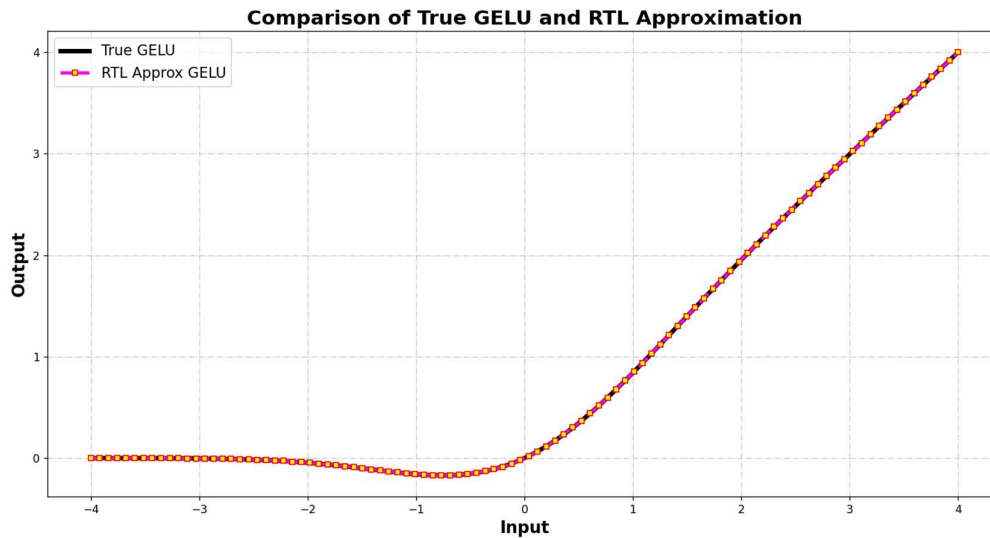


Figure 2: Comparison of True GELU and RTL Approximation

This graph compares:

- True mathematical GELU curve
- RTL-equivalent GELU approximation curve

Detailed Explanation:

1. The black curve represents the true mathematical GELU function.
2. The magenta dashed curve represents the RTL approximation.
3. Both curves overlap almost completely.
4. This indicates that the RTL design closely matches the actual GELU function.
5. The output smoothly increases for positive inputs.
6. Negative inputs are not completely removed, unlike ReLU.
7. The smooth nonlinear behavior confirms proper GELU implementation.

8. Small differences occur because of:

- Q8.8 fixed-point quantization
- LUT approximation
- rounding operations

Overall, this graph confirms that the RTL design accurately approximates the GELU activation function

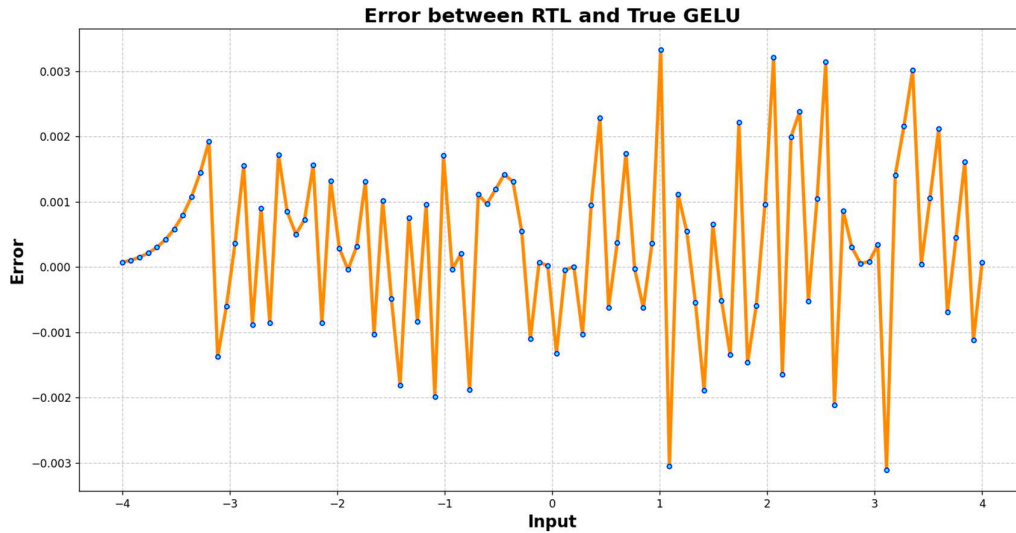


Figure 3: Error between RTL and True GELU

This graph shows the error between RTL output and true GELU output.

Error = RTL Output – True GELU Output

Detailed Explanation:

1. The error values remain very close to zero.
2. Maximum error is approximately ± 0.003 .
3. No large spikes are present.
4. Error fluctuates because of:
 - fixed-point rounding
 - LUT approximation
 - quantization effects
5. The error distribution is balanced around zero.
6. Positive and negative deviations are both very small.
7. This confirms that the approximation maintains good numerical accuracy.

Overall, the graph proves that the RTL implementation introduces only very small approximation error.

Verification Results:

Maximum Error = 0.003325

Mean Square Error (MSE) = 1.781791×10^{-6}

Interpretation:

- Maximum error is very small.
- MSE value is extremely low.
- The RTL approximation is highly accurate.
- The design is suitable for FPGA and ASIC implementations.

Conclusion: The Python verification successfully confirmed that the fixed-point RTL GELU implementation closely matches the mathematical GELU function.